

The Thiele Machine

A Mathematical Reference

Derived from the Machine-Checked Coq Corpus

Devon Thiele

Before the specification. I don't do math by hand. I think in shapes, directions, and connections, and my part of this work was to see where things had to fit, to direct, and to refuse what didn't fit. This document is the translation. Language models wrote the formal statements and the proofs, Coq checked every one of them, and I read the statements to make sure they say what the shape says. So nothing below is in my voice, on purpose. It's the reference layer: exact statements, exact premises, and where each one lives. If you want the pictures, the monograph starts with them. If you want to know exactly what is claimed, it's here.

The Thiele Machine is an abstract model of computation, and an argument made with it about what an account of computation should preserve.

It is a machine in the sense a Turing machine is one: a mathematical object, not a device. The model is the abstract `CertificationSystem` record and the `Substrate` interface. The small machine of `minimal/EarnedCore.v`, a two-counter machine whose commitments are earned, is one instance; the universal machine `U` built on it is another. They are witnesses, not the model.

Certification is one concrete witness of a broader investigation of structural state, observation, and cost. Selected windows lose distinctions that the richer state can express; cost laws constrain specified events under explicit hypotheses. A conventional encoding can preserve those distinctions. A window that discards them cannot recover them in general. The stronger proposal that these distinctions belong in a necessary foundation of computation is a conjecture. The status of each result is stated below.

Abstract

This is a mathematical reference for the Thiele Machine: a model of computation in which a step that raises a certification record must pay into a ledger that only grows. It presents the abstract interfaces, the main theorems, and the definitions of the small machine from the machine-checked Coq corpus. The Coq source is the complete definition; this document states each result with its premises and names the declaration that carries it.

Coq establishes that conclusions follow from premises. It does *not* establish that premises model physical reality. Every result is tagged as one of three types: **(S)** structural theorems about the model or a stated machine, **(C)** conditional results that hold under a named premise no machine fact discharges, and **(R)** refutations, where a theorem states the negation of a proposed claim and a named construction supplies the counterexample.

The core claims are structural (S): No Free Insight for every certification system; on finite machines, the toll derived from merge pricing for permanent certificates; the window and pricing results; the small machine's earned provenance, checker soundness and minimum price; Thiele completeness of the small machine and the failure of every clock-style machine; the window theorem for every Thiele-complete machine; the universal machine U; the record axis over any base; the recursion theorem and Rice's theorem for L; the classical CHSH bound; the algebraic Tsirelson bound; and, on the zero-marginal slice, the equivalence of column contractivity with positive semidefiniteness of one fixed 5×5 NPA matrix. Results stated over Landauer heat are conditional (C) on the named premise `landauer_heat`.

Contents

1	Introduction	2
1.1	Scope	2
2	Notation and Conventions	3
3	Certification Systems and No Free Insight	4
3.1	A2 among cost frameworks	4
3.2	Several floors and one charge	5
4	Finite Machines and the Price of Merging	6
4.1	Entropy and heat	7
4.2	An eight-state instance	7
4.3	No intrinsic joule value	7
5	Windows, Pricing and Narrowing	8
6	The Small Machine	9
7	Thiele Completeness	11
7.1	The window theorem	12
8	The Universal Machine	13
8.1	The universal host	13
8.2	U	13
9	Recursion and Undecidability	14
10	The Record Axis	15
10.1	Growing and probabilistic records	15
10.2	Bases	15
11	CHSH and Tsirelson	17
12	The Physical Boundary	18
13	External Systems and the Pointer Conjecture	19
13.1	The pointer criterion	19
A	Proof File Index	20
B	Epistemological Classification	21

Chapter 1

Introduction

A computation is usually described by what it reads and writes. The Thiele Machine adds a record of what the computation established and a ledger of what establishing it cost, and a rule tying the two together: the step that raises the record pays. The chapters below state that rule, derive its consequences, show which machines carry it, and prove where a window that hides the record and the ledger loses information.

Chapter 3 states the model and No Free Insight. Chapter 4 derives the toll on finite machines from the price of merging states. Chapter 5 states what a window can decide and price. Chapter 6 defines the small machine. Chapter 7 defines Thiele completeness and states the window theorem. Chapter 8 states the universal host and the universal machine U. Chapter 9 states the recursion and undecidability results. Chapter 10 states the record axis over any base. Chapter 11 states the CHSH and Tsirelson mathematics. Chapter 12 states the physical boundary. Chapter 13 states the models of external systems and the pointer conjecture.

1.1 Scope

Earlier releases of this repository carried a 51-instruction virtual machine and a hardware development. They are preserved at tag v3.3.0 and are not part of this reference. Nothing here depends on them.

Chapter 2

Notation and Conventions

- $\mathbb{N}$ is the natural numbers, $\mathbb{B} = \{\text{false}, \text{true}\}$, $\mathbb{R}$ the reals.
- A *trace* is a finite list of instructions. $\text{run}(\tau, s)$ folds the step function over τ from s .
- P is the certification reading, a function from states to $\mathbb{B}$. In Coq it is `cs_cert` on a `CertificationSystem`.
- $c(i) \in \mathbb{N}$ is the cost of instruction i ; $c(\tau)$ is the sum over a trace.
- μ is the ledger. Where a state carries it, conservation means $\mu(\text{step}(s, i)) = \mu(s) + c(i)$.
- A *window* is a function from states to observations.

Chapter 3

Certification Systems and No Free Insight

Definition 3.1 (Certification system). A certification system is a tuple $(S, I, \text{step}, c, P)$ with $\text{step} : S \times I \rightarrow S$, $c : I \rightarrow \mathbb{N}$, $P : S \rightarrow \mathbb{B}$, satisfying

$$\mathbf{A2}: \quad \forall s i, P(s) = \text{false} \wedge P(\text{step}(s, i)) = \text{true} \implies c(i) \geq 1.$$

Coq: the record `CertificationSystem`, with A2 as its field `cs_cert_costs`.

Theorem 3.1 (No Free Insight). *(S) In every certification system, if $P(s_0) = \text{false}$ and $P(\text{run}(\tau, s_0)) = \text{true}$, then $c(\tau) \geq 1$. Coq: `universal_nfi_any_substrate`.*

Proof. Induction on τ . Since P is false at the start and true at the end, some step in τ takes it from false to true. By A2 that step costs at least 1; every other step costs at least 0. $\square$

Theorem 3.2 (Simulating systems). *(S) Let a base certification system be simulated by a host: an embedding of base states into host states, a decoding of each base instruction into a list of host instructions, a commuting step law, and certification reflected by the embedding (record `SimulatingCertificationSystem`). A base trace from uncertified to certified costs at least 1 in the base, the host run of its decoded trace ends certified, and that host run costs at least 1. Coq: `host_represents_simulating_cert_system`.*

Theorem 3.3 (Quantitative form). *(S) Extend a certification system with a witness $w : S \rightarrow \mathbb{N}$ and a threshold k such that each step raises w by at most its cost and every certified state has $w \geq k$ (record `QuantitativeCertificationSystem`). A trace from a state with $w = 0$ to a certified state costs at least k . Coq: `universal_nfi_quantitative`.*

Theorem 3.4 (A2 is a restriction). *(S) A cost-bearing system without A2 can switch certification on for free, and A2 excludes exactly that. Coq: `honest_cost_tracking_strict_restriction`, `free_forgery_violates_A2`.*

3.1 A2 among cost frameworks

Theorem 3.5. *(S) A step system satisfies A2 if and only if every step has non-negative amortized cost under the potential that is 1 on uncertified states and 0 on certified ones; No Free Insight is then the potential method's telescoping bound. Coq: `a2_iff_nonnegative_amortized_cost`, `nfi_by_potential`. Under A2 the number of certification flips along a trace is at most its cost (`flips_le_cost`), and A2 together with the matching upper-bound condition is equivalent to exact pricing of a permanent certificate (`a2_and_aara_iff_exact`).*

These are local correspondences, not embeddings of the cited cost-framework families (graded monads, amortized resource analysis, cost translations, linear logic).

3.2 Several floors and one charge

Theorem 3.6 (Joint floors). *(S) For a finite family of events, the least cost schedule meeting every unit floor charges one on a step where any event occurs and zero otherwise. Coq: `joint_floor_is_least`. On pairs of Booleans, two independent coordinates that both rise in one step are met by a single charge of 1 (`independent_coordinates_joint_change_costs_one`).*

Independent state coordinates do not by themselves force additive cost.

Theorem 3.7 (Exact pricing). *(S) For trusted local-predicate pricing systems, a schedule that charges exactly one on the steps its predicate marks meets the certification floor and never overcharges exactly when its predicate marks precisely the certification flips. Coq: `exact_commitment_pricing_characterization`.*

Chapter 4

Finite Machines and the Price of Merging

Fix a state type S with a finite enumeration, instructions I , a step function and a reading P . The certificate is *permanent* when no step switches it off. An instruction is *injective* when its step function is.

Theorem 4.1 (A permanent flip merges). *(S) On a finite state space with a permanent certificate, an instruction that switches the certificate on at some state is not injective. Coq: permanent_flip_is_not_injective.*

Proof. If the instruction were injective it would map the certified states injectively into the certified states, so onto them, leaving no certified image for the uncertified state it switches on. $\square$

Theorem 4.2 (A2 from merge pricing). *(S) On a finite state space with a permanent certificate, if every non-injective instruction costs at least 1, A2 holds. Coq: a2_from_merging_price_and_permanence; the trace form is permanent_certification_trace_floor, and an honest erasure account gives the same bill (honest_erasure_accounting_implies_a2).*

Each premise is needed. Unbounded history escapes (`unbounded_history_escapes`), a revocable certificate escapes (`revocable_certificate_escapes`), and an unpriced merge escapes (`free_merge_escapes`).

Theorem 4.3 (The size of the bill). *(S) With m certified states and k uncertified states that one instruction switches on, the instruction collapses at least k states (`permanent_flips_collapse_at_least`). Priced one unit per halving of the number of distinct states, it costs at least $\log_2((m+k)/m)$. Coq: `permanent_flips_log_bound`.*

Theorem 4.4 (Which flips are forced). *(S) Over all schedules that price merges, an instruction is forced to have positive price if and only if it merges two states (`forced_priced_iff_merges`). An instruction that switches a reading on either merges or switches it off somewhere else (`flip_merges_or_revokes`). So every permanent-record write is forced, and the only free flips are revocable ones.*

Theorem 4.5 (Refutation). *(R) “The forced events are exactly the permanent-record writes” is false: a three-state instruction flips, revokes, and merges at once. Coq: `forced_price_without_permanent_record`.*

4.1 Entropy and heat

Theorem 4.6. *(S) After a permanent step, any distribution on the m certified states and the k flipping ones has Shannon entropy at most $\log_2 m$ (`permanent_step_entropy_ceiling`). From the uniform distribution the step removes at least $\log_2((m + k)/m)$ bits (`permanent_flip_uniform_entropy_drop`). An injective step removes none (`step_entropy_invariant_if_injective`).*

Theorem 4.7 (Heat). *(C) Given the named premise `landauer_heat`, which states Landauer’s principle for a step and a distribution, the flip dissipates at least $k_B T \ln((m + k)/m)$ from the uniform distribution (`permanent_flip_heat_floor`), and positive heat from any distribution that leaves every certified state and the flipping one possible (`permanent_flip_full_support_heat_positive`). For a state known in advance, the premise is equivalent to $\text{heat} \geq 0$, so it forces no heat (`known_state_flip_forces_no_heat`).*

4.2 An eight-state instance

Example 4.1. Four program slots and a certification flag make eight states. An advance step is injective (`fnext_injective`); the certify step and every jump merge states (`fcertify_merges`). The cost rule prices exactly the merges, so A2 holds there as a theorem (`fin_a2_from_merging_price`).

4.3 No intrinsic joule value

Theorem 4.8 (Refutation). *(R) For every state type and natural-number ledger, two real-valued scales agree on the ledger and differ in energy; the ledger fixes no energy unit. Coq: `mu_has_no_intrinsic_joule_value`. Choosing the scale $k_B T \ln 2$ gives one ledger unit that energy by definition (`calibrated_mu_landauer_energy`); the scale is a premise, not a measurement.*

Chapter 5

Windows, Pricing and Narrowing

Theorem 5.1 (Window adequacy). *(S) A query on states can be answered by a function of a window only if it is constant on the window’s fibers, the sets of states the window shows the same way (`decoding_requires_fiber_constancy`). Given a representative for every view, fiber constancy is equivalent to factoring through a decoder (`selected_representatives_give_decoder`).*

Theorem 5.2 (Shadow pricing). *(S) If a certifying step and a non-certifying step show the same window before and after, no price computed from the window meets the certification floor without charging the non-certifying step. Coq: `shadow_cannot_price_exactly`. A window that shows the reading prices certification exactly (`window_showing_reading_prices_exactly`).*

Theorem 5.3 (Retained history). *(S) Keeping the previous state in a history makes every deterministic step injective without changing its observed transition. Coq: `retained_history_step_injective`.*

Theorem 5.4 (Narrowing). *(S) On a finite machine that prices merges, each halving of the machine’s own set of possible states along a run is paid for (`run_narrowing_priced_log`). What an observer knows can shrink for free: a measurement that copies a hidden bit into a blank display forgets nothing, and one admissible merge-priced cost assigns it zero (`observer_narrowing_can_be_free`); wiping the display is a merge and costs at least one (`wipe_costs_at_least_one`).*

Chapter 6

The Small Machine

The small machine is defined in `minimal/EarnedCore.v` on the Coq standard library alone.

Definition 6.1 (State). A core has two counters $a, b \in \mathbb{N}$, their versions $v_a, v_b \in \mathbb{N}$, a program counter $pc \geq 1$, a table of facts, a commitment channel holding at most one fact, and a trap latch. A fact is a triple (property, counter, version); the properties are “is zero”, “is even” and “is at least n ”, each with a checker `eval` and a meaning `holds` tied by `eval_iff`. A state is a core with a ledger $\mu \in \mathbb{N}$ and a flag $P \in \mathbb{B}$. The table holds at most 16 facts. The start state for (a, b) has no facts, an empty channel, the flag down and $\mu = 0$.

Definition 6.2 (Instructions and costs). `INC c` adds one to counter c . `DEC c j` subtracts one and jumps to j if $c > 0$, and otherwise falls through. `HALT` stops. `CHECK p c` evaluates p on counter c and, if it holds and the table has room, writes the fact (p, c, v_c) . `COMMIT p c` succeeds only if the fact (p, c, v_c) is in the table, and then sets the channel to it. `CERTIFY` succeeds only if the channel names a fact. Every failure sets the trap latch, after which no instruction changes anything but the ledger. Every write to a counter adds one to its version. `CHECK`, `COMMIT` and `CERTIFY` cost 1; the others cost 0. The ledger adds the cost of every executed instruction, and the flag rises exactly on a passing `CERTIFY`.

Theorem 6.1 (Universality). *(S) The INC/DEC fragment is the two-counter machine step for step, and a program halts on the small machine exactly when it halts as a two-counter program. Coq: `simulation_run`, `halting_correspondence`. Its halting problem is undecidable, by reduction from the vendored two-counter result (`earned_core_halting_undecidable`). The Turing-universality of two-counter machines is Minsky’s theorem, cited.*

Theorem 6.2 (The toll). *(S) The ledger is the exact sum of executed costs (`mu_conservation_trace`); only `CERTIFY` raises the flag (`only_certify_certifies`); the flag never drops (`cert_permanent`); A2 holds (`a2`); and a run from an uncertified state to a certified one pays at least 1 (`nfi_floor`).*

Theorem 6.3 (Earned provenance). *(S) From a clean start, every certified run contains a passing `CHECK` of some claim, then a passing `COMMIT` of the same claim with that counter untouched in between, then `CERTIFY`. Coq: `earned_certification_provenance`.*

Theorem 6.4 (Checker soundness and no forging). *(S) On a run from a clean start, a fact whose counter has not been written since its check names a true claim about that counter (`checker_soundness`), and every fact in the table was written by a passing `CHECK` of exactly that claim (`no_forging`).*

Theorem 6.5 (Price). *(S) From a clean start a certified run costs at least 3 (`certified_run_min_cost`), and a three-instruction program pays exactly 3 (`min_cost_tight`).*

Theorem 6.6 (A check that can fail). *(S) The program CHECK “a is zero”, COMMIT, CERTIFY certifies from a start with $a = 0$, paying exactly 3, and from any other start traps with the flag down, for every number of steps. Coq: `earned_run_check_can_fail`, `earned_run_refused_forever`.*

Theorem 6.7 (Links). *(S) The small machine is a certification system with the trace floor (`earned_core_floor`), and as a record-carrying machine its record moves as a latch on one base event (`earned_core_is_latch`).*

Remark 6.1. `minimal/EarnedGeneric.v` states the same machine over any property language with an exact checker; sorted lists are one instance (`sorted_cs_sound`). `minimal/EarnedMulti.v` has a counter for every natural number and is the host machine of Chapter 8.

Chapter 7

Thiele Completeness

Definition 7.1 (Machine, weak completeness). A machine is a state type, a move type, a step, a cost per move and a record $P : S \rightarrow \mathbb{B}$ (record `machine`). It is weakly Thiele-complete when it is a certification system whose base is universal.

The weak notion is nearly empty: a two-counter machine that charges 1 for every move meets it for any record (`clock_weakly_thiele_complete`).

Definition 7.2 (Thiele complete). A machine is Thiele-complete when its moves can be read as base moves and three record moves, CHECK, COMMIT and CERTIFY of a claim, over a type of claims with a meaning on states, a checker, a notion of the thing a claim is about being unchanged, a set of clean starts and a ledger, such that:

- (a) *Universal base*. Every two-counter instruction has a base move that acts on a window of the state exactly as the instruction acts on a two-counter configuration, every two-counter start has a clean live state showing it, base moves never change the record, and the record once up stays up.
- (b) *Earned record*. A clean start has the record down. Every run from a clean start that ends with the record up contains a passing CHECK of some claim, then a COMMIT of the same claim with the thing it is about unchanged in between, then the CERTIFY that raises the record. A passing check means its claim holds, and a claim that holds keeps holding while the thing it is about is unchanged.
- (c) *Exact toll*. Base moves cost 0, each record move costs exactly 1, and the ledger grows by the cost of each move.
- (d) *Non-vacuity*. Some claim is true at some clean start and false at another, and the bare chain CHECK, COMMIT, CERTIFY on it raises the record from a clean start exactly when the claim holds there.

Coq: `thiele_complete` in `minimal/ThieleComplete.v`.

Theorem 7.1 (Consequences). *(S) In every Thiele-complete machine: the ledger counts the record moves (`ledger_counts_record_moves`); a certificate costs at least 3 (`certificate_costs_three`); a committed claim holds when committed (`committed_claim_holds`); some check fails (`check_can_fail`); some run certifies (`some_run_certifies`); the move raising the record from a clean start is a CERTIFY (`only_certify_raises`); and some move costs 0 (`complete_has_free_move`).*

Theorem 7.2 (Machines that meet it). *(S) The small machine is Thiele-complete (`earned_core_thiele_complete`), and so are the machine over any property language with an exact checker and a property true of one value and*

false of another (*earned_generic_thiele_complete*) and the sorted-list machine (*sorted_machine_thiele_complete*).

Theorem 7.3 (Machines that fail it). *(R)* Every clock-style machine, with any record, is weakly Thiele-complete and not Thiele-complete (*clock_not_thiele_complete*); so is a clock whose record is a latch on its configuration (*latch_clock_not_thiele_complete*). A machine with a free base and one paid move that raises a latch meets clauses (a) and (c) and fails (b) (*one_move_record_excluded*, *paid_latch_not_thiele_complete*). A machine whose record never rises meets (a), (b) and (c) and fails (d) (*silent_not_thiele_complete*).

7.1 The window theorem

Theorem 7.4 (Every Thiele-complete machine hides its record). *(S)* For every Thiele-complete machine there are a clean start and two runs from it, the second made of base moves only, that end at the same base window; the first ends certified with its ledger at least 3 above the start, and the second ends uncertified with its ledger unchanged (*complete_two_runs*). Hence no function of the base window recovers the record (*complete_hides_record*) or the ledger (*complete_hides_ledger*) on every reachable state (*thiele_complete_hides*).

Proof. Clause (d) and clause (b) give a clean start and a certifying run. The base is universal, so base moves alone reach the window the certifying run ends at; base moves change neither the record nor the ledger. $\square$

Every Thiele-complete machine is a certification system with the trace floor, and its window is blind to its reading (*thiele_complete_floor*, *complete_cs_window_blind*).

Chapter 8

The Universal Machine

8.1 The universal host

Theorem 8.1. *(S) The host of `minimal/UniversalThiele.v` runs any small-machine program as a guest. Its mirror of the guest's flag always equals that flag (`record_agreement`). The mirror rises only in a guest step that passed `CERTIFY`, charged in that same host step (`no_free_host_certification_step`, `toll_enforced_by_host_step`). The host is a certification system with the trace floor (`host_nfi`).*

A machine from elsewhere enters through the two-counter encoding; its computation and the toll are kept, its own price list is not.

8.2 U

U is one fixed program of the machine of `minimal/EarnedMulti.v`, with the single property of `minimal/UniversalCodes.v`, made of the small machine's own instruction kinds and with no guest step built into anything (`coq/kernel/foundation/UniversalLayout.v`). For a small-machine program P and a start (x, y) , g_m is the guest after m steps and h_n is U after n steps from the loaded state.

Theorem 8.2 (Simulation). *(S) For every m there is $n \geq m$ with h_n related to g_m by the simulation relation, or both machines have stopped. Coq: `U_simulation`.*

Theorem 8.3 (Halting and output). *(S) The guest halts if and only if U halts (`universal_halting`); at halting, U's first two registers hold the guest's counters, and the trap latches, ledgers and flags are equal (`universal_output`).*

Theorem 8.4 (The flag is earned again). *(S) The guest's flag ever rises if and only if U's flag ever rises (`universal_flag_iff`). A raised flag of U stands on U's own passing `CHECK`, its own `COMMIT` of the same claim with the slot untouched, and the `CERTIFY` that raised it; when checked, the slot held the guest's claim and value, and the guest's own run contains its chain on the same claim (`universal_earned`).*

Theorem 8.5 (The ledger). *(S) At matching points U's ledger equals the guest's, and every ledger of U lies between the guest's ledger at some m and at $m + 1$. Coq: `universal_ledger_exact`.*

Theorem 8.6. *(S) The machine U runs on, read with `INC` and `DEC` as its base and `CHECK`, `COMMIT` and `CERTIFY` as its record moves, is Thiele-complete (`universal_thiele_complete`). Its halting problem is undecidable (`interp_halting_undecidable`).*

Chapter 9

Recursion and Undecidability

Definition 9.1 (Substrate). A substrate supplies states, programs, a partial run function, a ledger that no step decreases, an encoding of programs into states with a decoder, behavioral equivalence (equal runs from every state), a representability predicate on program transformers, and Kleene’s recursion theorem for representable transformers as a field (class `Substrate`). A shortcut predicate on programs respects behavioral equivalence and has one accepted and one rejected canonical program (class `WithShortcutPredicate`).

Theorem 9.1 (Structural undecidability). *(S) In every substrate with a shortcut predicate, no decider whose flip transformer is representable returns true exactly when the predicate holds. Coq: `structural_shortcut_undecidable`.*

Proof. Let D be such a decider and $f(p)$ the rejected canonical program if $D(p)$ is true and the accepted one otherwise. The recursion theorem gives q behaviorally equivalent to $f(q)$. If $D(q)$ is true, q behaves like the rejected program, so the predicate fails at q ; if false, it holds. Either way D is wrong at q . $\square$

The natural-number substrate meets the recursion premise by construction, since its fixed-point program copies the transformer’s output (`nat_structural_shortcut_undecidable`).

Theorem 9.2 (L). *(S) For the weak call-by-value lambda calculus L , defined in `coq/kernel/foundation/LRecursion.v` from its reduction rules, Kleene’s second recursion theorem holds (`second_recursion`), every extensional nontrivial property of L terms is undecidable (`L_rice`), and L halting is undecidable (`L_halting_undecidable`). The Turing-completeness of L is cited from Forster and Smolka, not re-proved.*

Theorem 9.3. *(S) The complement of two-counter halting is undecidable, composed from the vendored library’s reductions (`MM2_HALTING_compl_undec`).*

The diagonal uses recursion and behavioral equivalence; cost plays no part. Deciding a predicate over arbitrary programs is a different task from reading one field of a given state.

Chapter 10

The Record Axis

Definition 10.1 (Bases and honest extensions). A base is a deterministic record-free machine: states, a next function, initial states and halting (record `BaseMachine`). A record-carrying machine has states, a next function, initial states, a record, a ledger and halting (record `RCM`). A cover maps it onto a base step for step, preserving initial states and halting. The extension is *honest* when its record's next value depends only on the base state and itself, the ledger never decreases, A2 holds, the record is permanent, and some reachable step writes it (`HonestBaseExtension`).

Theorem 10.1 (The record axis is a latch). *(S) Every honest extension of every base factors as a latch: there is a base event h such that the base state and the record evolve as $(b, r) \mapsto (\text{next}(b), r \vee h(b))$. Coq: `record_axis_is_latch_holds`. Two permanent records factor as two latches, each reading the other (`record_pair_is_two_latches_holds`).*

Theorem 10.2 (Both premises do work). *(R) A record that can switch back off is not a latch (`toggle_not_latch`). A record switched on by a hidden clock is permanent and not driven by the computation (`clock_record_not_driven`).*

Theorem 10.3 (Which machines carry the axis). *(S) Every base carries it: the base plus a latch on any event it reaches, charging one unit when the latch sets, is an honest extension (`latch_core_honest`). A reversible base with unbounded memory carries it reversibly (`history_latch_honest`). With finite memory, a step that writes a permanent record is not injective (`finite_reversible_cannot_write`).*

10.1 Growing and probabilistic records

Theorem 10.4. *(S) A growing record in a partial order satisfies a complete family of threshold-latch equations and keeps its price schedule (`growing_record_decomposes_holds`). A monotone chain of pairwise-distinct k -bit vectors has at most $k + 1$ members (`chain_needs_bits_holds`).*

Theorem 10.5. *(R) One Boolean latch does not carry every growing record: a three-value chain over a one-state base is a counterexample (`one_latch_refuted`). In a finite branching model a deterministic latch does not handle branching (`deterministic_latch_handles_branching_refuted`), and equal branch support and charges do not determine the branch weights (`schedule_determines_probabilities_refuted`).*

10.2 Bases

The relation `weak_base_equiv`, agreement on chosen observations and halting with stuttering steps, is an equivalence (`weak_base_equiv_trans_holds`). The record axis is a

latch over the Cook and Reckhow RAM and over L (`record_axis_is_latch_on_ram_holds`, `record_axis_is_latch_on_l_holds`); these instances follow from the general theorem and do not distinguish the bases. Cost-respecting simulations between different bases are not constructed.

Chapter 11

CHSH and Tsirelson

The results in this chapter are mathematics about correlators and trial counts. No machine is fixed.

Theorem 11.1 (Classical bound). *(S) A factorizable correlator has $|S| \leq 2$ (`factorizable_CHSH_classical_bound`); trial counts consistent with one deterministic local strategy cannot exceed the classical bound (`chsh_stat_violation_not_local`).*

Theorem 11.2 (Algebraic Tsirelson bound). *(S) A rational correlator satisfying the absolute-value bounds and four selected 3×3 polynomial minors has $|S| \leq 2\sqrt{2}$ (`algebraically_coherent_tsirelson_general`). The bound also follows from row constraints and Cauchy–Schwarz (`tsirelson_from_row_bounds`) and is reached (`tsirelson_tight`). No Hilbert space is used.*

Theorem 11.3 (The biconditional). *(S) On the zero-marginal slice, the column-contractivity conditions on $E_{00}, E_{01}, E_{10}, E_{11}$ hold if and only if the zero-marginal 5×5 NPA moment matrix is positive semidefinite. Coq: `column_contractive_iff_npa_psd`.*

Theorem 11.4 (The integer check). *(S) If the integer column check accepts eight trial counts, the zero-marginal NPA matrix built from their correlators is positive semidefinite (`column_contractive_check_witness_npa_psd`), and a PSD matrix of that form satisfies the Tsirelson bound (`npa_psd_implies_tsirelson_bound`). The check is sound in this direction only; a rejection is not proved to mean the conditions fail.*

Theorem 11.5 (Elliptope). *(S) The CHSH correlator quantum set is an existential completion of the zero-marginal NPA matrix; points of it satisfy the Tsirelson bound (`elliptope_tsirelson`), and the PR box is outside it (`pr_box_not_elliptope`). An integer membership check is sound into realizability (`elliptope_check_full_sound`).*

No theorem here identifies a correlator with a physical experiment or proves that any device produces CHSH-violating statistics.

Chapter 12

The Physical Boundary

Theorem 12.1 (Calorimeter). *(S) In a two-state calorimeter protocol, a canonical reset from half-excited to ground satisfies a discrete master equation exactly, and at fixed Hamiltonian the bath receives $\Delta/2$ (`canonical_reset_heat_exact`).*

Theorem 12.2 (Refutation). *(R) Population dynamics do not fix heat: two gaps with identical dynamics release different heat (`master_equation_does_not_fix_heat_scale`).*

Ruling a gap out needs a thermal premise such as local detailed balance, or device evidence. Converting the ledger to energy requires thermal and device premises; the ledger has no intrinsic energy unit (Chapter 4).

Chapter 13

External Systems and the Pointer Conjecture

Each model below is narrow. None is a full security, consensus, database or audit-log theorem.

- **RFC 9162.** The iterative inclusion and consistency checks of the specification reject the boundary cases and accept the specification’s examples under symbolic digests (`rfc9162_inclusion_boundary_safe`, `rfc9162_consistency_boundary_safe`). Collision resistance and signed-tree-head authenticity are cryptographic premises, not proved.
- **Proof-carrying code.** A toy checker accepts exactly when the address-bound verification condition holds (`pcc_checker_accepts_iff_vc`). It mirrors the roles of Necula’s system; it is not that system.
- **TPM quotes.** A bare sign/verify interface does not imply quote authenticity (`tpm_interface_authenticity_refuted`).
- **Casper FFG.** Two finalized blocks on conflicting branches imply a slashable quorum (`accountable_safety`). This is the known accountable-safety theorem.

The addressed-RAM and reversible-update files prove only their named fragments. The cost-semantics correspondences are not full cost-framework embeddings.

13.1 The pointer criterion

The pointer-observable criterion proposes that the event a step rule is forced to price is singled out by redundant record proliferation across observers. Twelve candidate events over six observer maps proliferate as stated (`twelve_candidate_measurements_checked`). The MAC-labelled model does not proliferate (`mac_model_not_proliferating`); applying that to a real system depends on the observer model. A two-observer toggle game with consensus, authenticity and coordinator-free updates revokes its event (`toggle_game_refutes_strong_pointer_necessity`), so those properties do not force permanence. Durable views do imply permanence, by making persistence a premise (`durable_consensus_implies_permanence`). The external consequence survey contains five narrow statements already known from their governing specifications and five stronger statements without enough protocol or hardware semantics; no surveyed consequence is simultaneously record-axis dependent, novel, and ready for use. The pointer criterion is a conjecture.

Appendix A

Proof File Index

Results	File
No Free Insight, simulating systems	coq/kernel/nfi/UniversalCertificationCost.v
Quantitative form	coq/kernel/nfi/QuantitativeNoFI.v
Cost frameworks	coq/kernel/nfi/CostSemanticsComparison.v, coq/kernel/nfi/CostFrameworks.v
Permanent certificates, merge pricing	coq/kernel/nfi/PermanentCertification.v, coq/kernel/nfi/PermanentRecordPricing.v
Entropy and heat	coq/kernel/nfi/PermanentCertificationEntropy.v
Eight-state instance	coq/kernel/nfi/FiniteCertMachine.v
Shadow pricing	coq/kernel/nfi/ShadowPricing.v
Narrowing	coq/kernel/nfi/KnowledgeNarrowing.v
Window adequacy, joint floors	coq/kernel/frontier/ObservationPolicy.v
The small machine	minimal/EarnedCore.v, coq/kernel/foundation/EarnedCoreLinks.v
Thiele completeness	minimal/ThieleComplete.v, coq/kernel/foundation/EarnedGenericLinks.v
The window theorem	minimal/ThieleCompleteWindow.v
Universal host	minimal/UniversalThiele.v, coq/kernel/foundation/UniversalThieleLinks.v
U	coq/kernel/foundation/UniversalRun.v, coq/kernel/foundation/UniversalInterpreterLinks.v
Structural undecidability	coq/kernel/nfi/StructuralUndecidability.v, coq/kernel/foundation/NatSubstrateInstance.v
L recursion and Rice	coq/kernel/foundation/LRecursion.v
Record axis	coq/kernel/foundation/StructuralRecordAxis.v, coq/kernel/foundation/RecordAxisDiscrimination.v
Growing and probabilistic records	coq/kernel/foundation/GrowingRecord.v, coq/kernel/foundation/ProbabilisticRecord.v
CHSH and Tsirelson	coq/kernel/quantum/CHSHColumnCheck.v, coq/kernel/quantum/TsirelsonGeneral.v, coq/kernel/category/AlgebraicCoherence.v
Calorimeter	coq/kernel/thermodynamic/CalorimeterProtocol.v
Pointer criterion	coq/kernel/frontier/PointerObservable.v, coq/kernel/frontier/RecordProliferationSurvey.v, coq/kernel/frontier/EcosystemGame.v

Appendix B

Epistemological Classification

Result	Status	Key premise
No Free Insight	(S)	A2 as a field of the record
A2 from merge pricing	(S)	Finite states, permanent certificate, merge price
Heat floor	(C)	<code>landauer_heat</code>
No intrinsic joule value	(R)	None beyond the ledger
Shadow pricing	(S)	A colliding pair in the window
Earned provenance, checker soundness	(S)	Clean start
Thiele completeness of the small machine	(S)	The four clauses
Clock-style machines fail	(R)	The four clauses
Window theorem	(S)	Thiele completeness
U	(S)	Small-machine programs
Structural undecidability	(S)	Recursion theorem of the substrate
L recursion and Rice	(S)	L's reduction rules
Record axis is a latch	(S)	Honest extension
Algebraic Tsirelson bound	(S)	Polynomial minors
Calorimeter heat scale	(R)	Fixed dynamics
Pointer criterion	Conjecture	Observer model